

Análisis Profundo del Software

Alcance de esta revisión

Este análisis fue realizado sobre la base real del proyecto: estructura de carpetas, código fuente, empaquetado, activos visuales, documentación y archivos operativos presentes en el repositorio.

No modifiqué el código ni ejecuté el producto en modo interactivo. Por lo tanto, esta opinión está basada en inspección técnica profunda del software y no en una demo visual en tiempo real.

Veredicto Honesto

Este software no se ve como un prototipo improvisado. Se nota trabajo serio, intención de producto, conocimiento del problema operativo y varias capas que ya superan a muchas herramientas internas hechas "para salir del paso".

Mi opinión sincera es esta:

- El proyecto está bastante avanzado en funcionalidad y en visión.
- El producto ya tiene identidad, propuesta clara y utilidades reales para operación de terreno.
- La interfaz está bastante mejor que la media de una app interna en Python con Tk.
- La deuda principal no está en "falta de funciones", sino en coherencia arquitectónica, cierre de migraciones y madurez de producto.

En una frase: tienes una herramienta valiosa y con personalidad, pero hoy conviven un software real y útil con una refactorización incompleta que todavía no termina de convertirse en una base sólida y mantenible.

Resumen Ejecutivo

Lo mejor del software

- Tiene una propuesta concreta y útil: monitoreo black-box de equipos por conectividad, con foco operacional real.
- La funcionalidad principal está bien pensada para el contexto: ping, historial, uptime, alertas, usuarios, diagnósticos, reportes y backups.
- La UI no es fea ni genérica; tiene intención, branding y lenguaje visual de herramienta profesional.
- Hay una base de datos SQLite, alertas por Telegram, autenticación, empaquetado con PyInstaller y un instalador Inno Setup.
- La documentación es abundante y demuestra que el proyecto fue pensado como producto, no solo como script.

Lo más delicado

- La aplicación tiene dos arquitecturas conviviendo al mismo tiempo: una monolítica activa y una modular nueva que todavía no es la que arranca `main.py`.

- Hay inconsistencias de identidad y versión: "Argos Guard", "Anvic Network Monitor" y "Sentinel" aparecen mezclados.
- La rama modular nueva, aunque más ordenada, en algunos puntos parece menos segura que la rama activa.
- Existen señales de trabajo no cerrado: conflicto de merge en `installer.iss`, legado duplicado, datos operativos mezclados con el proyecto y varios caminos de persistencia distintos.
- No vi una suite de pruebas automatizadas ni una capa clara de validación integral.

Mi diagnóstico global

El software ya sirve y tiene bastante valor. No lo veo verde. Lo veo en etapa "pre-producto serio" o "producto interno potente", pero todavía no en estado de "base limpia y confiable para crecer sin miedo".

Si hoy lo usas tú o un grupo pequeño, va por buen camino. Si quieres que esto escale, se herede, se venda o se mantenga por años, la prioridad ya no es agregar más cosas: es consolidar el núcleo.

Infraestructura y Arquitectura

1. Tipo de solución

La app es una solución de escritorio Windows-first construida en Python con `customtkinter`, enfocada en monitoreo local de red, operación manual y empaquetado a ejecutable.

Pila observada:

- UI: `customtkinter`, `tkinter`, `matplotlib`
- Monitoreo: `ping`, `arp`, `tracert`, `nslookup`, `netstat`, `sc`, `net`
- Persistencia: SQLite y archivos JSON/JSON cifrados
- Alertas: Telegram vía HTTP
- Audio: `pygame`
- Reportes: `reportlab`
- Distribución: PyInstaller + Inno Setup

Esta elección tecnológica tiene sentido para tu caso real:

- opera en LAN
- depende de Windows
- necesita interfaz visual local
- debe correr sin infraestructura compleja

No intentaste sobrediseñarlo como sistema distribuido cuando el problema era local y operativo. Eso es una buena decisión.

2. Hallazgo arquitectónico más importante

El proyecto tiene dos generaciones activas al mismo tiempo:

Arquitectura activa hoy

`main.py` sigue arrancando:

```
from monitor import App
```

Eso significa que la aplicación real hoy depende todavía del monolito `monitor.py`.

Arquitectura objetivo/refactorizada

Ya existe una estructura mucho más sana:

- `core/`
- `ui/`
- `services/`
- `database/`
- `auth/`
- `utils/`

Esa estructura nueva está mejor pensada y mejor separada, pero no es el entrypoint real.

Mi lectura sincera

Esto no es un detalle menor. Es el corazón del diagnóstico.

Hoy tu proyecto parece estar en transición entre:

1. un sistema que ya funciona en producción local
2. una refactorización que va en la dirección correcta

El problema es que ambas realidades conviven y eso genera ruido técnico:

- código duplicado
- comportamientos distintos
- seguridad distinta según la rama
- branding distinto
- persistencia distinta
- riesgo de que alguien crea que la app usa la arquitectura nueva cuando en verdad sigue usando la antigua

Mi juicio aquí es claro:

La refactorización es prometedora, pero hoy todavía no está consolidada.

3. Calidad de modularización

La arquitectura nueva tiene muy buenas decisiones:

- `services/network_tools.py` separa lógica de red de la UI
- `database/database_manager.py` encapsula SQLite
- `ui/tabs/*` organiza la interfaz por contexto funcional
- `utils/paths.py` centraliza rutas para PyInstaller
- `core/backup_manager.py` separa respaldos del resto

Eso habla bien de tu criterio técnico.

Pero esa buena intención queda debilitada por:

- coexistencia con módulos raíz antiguos que hacen casi lo mismo
- presencia de carpeta `legacy/`
- diferencias entre la lógica nueva y la lógica realmente ejecutada

Conclusión: la modularidad existe, pero todavía no gobierna el producto.

Seguridad y Persistencia

1. La rama activa tiene decisiones de seguridad mejores que la rama nueva

Esto fue uno de los hallazgos más importantes.

En la rama activa

Se observa:

- cifrado de configuración con `Fernet`
- clave maestra protegida con DPAPI en Windows
- usuarios cifrados en `.enc`
- hashing de contraseñas con `bcrypt`

Eso, para una app de escritorio interna, está bastante bien.

En la rama modular nueva

Se observa:

- usuarios en `users.json`
- hash SHA-256 simple
- configuración de equipos en JSON plano

Esto, comparado con la rama activa, es un retroceso.

Opinión directa

Si mañana cambiaras el entrypoint al sistema modular sin revisar esto, podrías mejorar arquitectura pero empeorar seguridad.

Ese tipo de regresión silenciosa es exactamente el tipo de "error fantasma" que después pega fuerte porque nadie lo nota hasta que ya migró todo.

2. Persistencia fragmentada

Hoy aparecen varios mecanismos coexistiendo:

- `anvic_monitor.db`
- `argos_guard.db`
- JSON

- JSON cifrado `.enc`
- archivos `.corrupt.*`
- `.bak`
- reportes PDF generados

Esto revela evolución, pero también deja una sensación de sistema con varias capas superpuestas.

No es caótico, pero sí heterogéneo.

Riesgos de esta fragmentación

- migraciones confusas
- rutas difíciles de seguir
- soporte más complejo
- posibilidad de leer/escribir en formatos distintos según la rama usada
- dificultad para respaldar o restaurar con certeza

Mi recomendación conceptual es clara:

el sistema necesita una sola historia oficial para usuarios, configuración y métricas.

Funcionalidades

1. Monitoreo principal

La funcionalidad núcleo está bien resuelta para tu contexto:

- monitoreo por IP
- estado conectado/desconectado
- lectura de MAC vía ARP
- intervalo configurable
- tarjetas por equipo
- recarga manual
- detección y conteo de desconexiones

Esto está alineado con un monitoreo operativo realista cuando no tienes APIs, agentes ni SNMP.

No estás intentando vender humo técnico. Estás resolviendo el problema que sí puedes resolver.

2. Historial y analítica

Aquí el software sube mucho de nivel respecto a un monitor básico:

- latencia histórica
- uptime
- heatmap de disponibilidad
- gauges de estado
- tabla de eventos

Esto le da valor real porque deja de ser solo "ver si algo está arriba o abajo" y pasa a ser una herramienta para detectar patrones, justificar reclamos y construir evidencia.

Punto fuerte importante:

la app entiende bien su rol de evidencia operativa.

3. Alertas

Tienes tres capas de alerta:

- visual
- sonora
- Telegram

Eso es muy bueno para un entorno operacional.

Además, la restricción horaria para Telegram me parece una decisión madura: no todo sistema necesita spamear 24/7 si el proceso real no funciona así.

4. Diagnóstico y utilidades de red

La pestaña de diagnóstico tiene bastante valor práctico:

- ping
- tracer
- nslookup
- ARP
- netstat
- estado de servicios
- acciones de mantenimiento de red

Esto vuelve la app más completa y reduce el cambio de contexto del operador.

Mi lectura:

no es solo monitor; empieza a parecer una consola de operación de red liviana.

5. Gestión de usuarios

Hay roles, login, setup inicial y cambio forzado de contraseña.

Para una app interna Windows de este tipo, ya es una capa de madurez importante.

No está al nivel de un IAM serio, pero cumple muy bien para el tamaño del producto.

6. Reportes y backups

La generación de PDF y la presencia de backups automáticos/manuales son dos señales claras de producto útil en contexto real.

Esto tiene valor de negocio y valor político:

- sirve para evidenciar
- sirve para presentar
- sirve para respaldar operación

Eso está bien pensado.

Apariencia, UX y Producto

1. Apariencia general

La apariencia tiene carácter.

Se nota una intención tipo:

- NOC liviano
- dashboard técnico
- herramienta de vigilancia operativa

Lo que transmite visualmente es:

- fondo oscuro
- acentos cian
- estado verde/rojo claro
- tarjetas
- tabs
- métricas visuales
- branding tecnológico

No se siente como formulario gris de oficina. Eso suma mucho.

2. Lo que está bien visualmente

- El uso de `customtkinter` le da una presencia más moderna que Tk puro.
- Las tarjetas de equipos son una buena decisión de lectura rápida.
- Los colores de estado están bien elegidos.
- Las pestañas separan bien monitoreo, historial, diagnóstico y administración.
- Los toasts y sonidos le dan sensación de sistema vivo.
- Los gráficos elevan mucho la percepción de valor del producto.

3. Lo que hoy se siente menos fino

Aunque la app se ve mejor que muchas herramientas internas, todavía hay cosas que la hacen sentir "muy buena app técnica" más que "producto visual completamente pulido":

- uso abundante de emojis en UI y textos
- tipografía muy estándar (`Arial`)
- tamaños y geometrías bastante fijos
- layout algo rígido
- alta dependencia de resoluciones concretas

- mezcla de estilos entre ventanas

No diría que la app es fea. Para nada. Diría que está visualmente por encima del promedio, pero aún no totalmente refinada.

4. Problema de identidad visual

Aquí sí hay una incoherencia visible:

- logos de Argos Guard
- logo Anvic Security
- nombres "Sentinel", "Argos Guard" y "Anvic Network Monitor"
- versiones 2.1, 2.5 y referencias mezcladas

Eso debilita la percepción de madurez.

No es un problema cosmético menor: la identidad partida también suele reflejar código partido.

Mi opinión brutalmente honesta:

el producto tiene mejor diseño operativo que diseño de marca.

Fortalezas Reales del Proyecto

Estas son, para mí, las fortalezas más sólidas:

1. Entiende muy bien el problema de negocio

No es una app inflada. Está hecha para un escenario concreto y eso se nota en casi todas las decisiones.

2. Tiene visión de producto

No te quedaste en "hacer ping y pintar verde/rojo". Agregaste historial, métricas, alertas, reportes, roles, diagnóstico y empaquetado.

3. La UX está pensada para operación

El operador no necesita abrir cinco herramientas distintas para entender qué pasa.

4. Hay señales sanas de evolución técnica

La existencia de la arquitectura modular demuestra que ya detectaste los límites del monolito y empezaste a corregirlos.

5. La documentación es abundante

Eso ayuda mucho a mantener criterio, visión y contexto, especialmente en proyectos que nacen de una necesidad real y van creciendo por fases.

Debilidades y Riesgos Ocultos

Aquí va la parte sin anestesia.

1. La migración arquitectónica está a medio cerrar

Este es el problema más serio.

Tener:

- `monitor.py` como núcleo real
- `core/app.py` como nueva base
- `legacy/` todavía presente
- duplicación de auth, ping, metrics y network tools

es una receta clásica para deuda acumulada.

No porque el software esté mal, sino porque deja demasiadas verdades paralelas.

2. Riesgo de regresión al migrar

La rama modular nueva se ve más limpia, pero no necesariamente más madura en todo.

Ejemplos:

- seguridad de usuarios más débil
- configuración en plano
- branding inconsistente
- base de datos distinta

Eso significa que migrar “por orden” podría romper cosas reales.

3. Higiene operativa mejorable

En el árbol actual aparecen archivos que no deberían mezclarse tan cerca del corazón del proyecto:

- archivos `.enc`
- archivos `.corrupt.*`
- `.bak`
- PDF generado
- bases SQLite
- carpetas de build y salida presentes en el workspace

Además, `installer.iss` tiene conflicto de merge sin resolver.

Eso no mata la app, pero sí baja mucho la percepción de control del proyecto.

4. Falta de una estrategia de pruebas visible

No encontré una suite de pruebas automatizadas clara.

En un producto así, los bugs más peligrosos no siempre son sintaxis:

- hilos colgados

- toasts disparados de más
- duplicación de monitores
- problemas de persistencia
- pérdida de compatibilidad entre formatos
- diferencias entre desarrollo y ejecutable

Sin pruebas o checklist fuerte, esos errores se escapan fácil.

5. Validaciones funcionales aún incompletas

El alta de equipos, según la rama, no siempre valida IP/dominio con suficiente rigor antes de guardar.

Eso puede producir:

- configuraciones sucias
- errores silenciosos
- equipos inválidos persistidos

6. Windows-dependencia muy fuerte

En tu contexto actual no es malo. De hecho, probablemente es correcto.

Pero sí limita:

- portabilidad
- despliegue futuro
- uso fuera del equipo principal
- automatización más neutral

No lo trataría como fallo hoy. Sí como limitación estratégica.

“Errores Fantasma” o Señales que Merecen Atención

Estas son las señales más finas que encontré y que vale oro tener mapeadas:

1. El entrypoint real no usa la arquitectura nueva

Esto puede generar falsa sensación de refactor terminado cuando aún no lo está.

2. La seguridad de la rama nueva parece más débil que la actual

Muy importante. Si no se controla, una migración puede introducir downgrade sin que nadie lo note.

3. Branding y assets no están totalmente alineados

Hay referencias a íconos/nombres/versiones que no parecen pertenecer a una única línea de producto cerrada.

4. `installer.iss` tiene conflicto de merge

Esto es una bandera roja de mantenimiento y release.

5. Hay varias fuentes de verdad

Usuarios, configuración, branding, métricas y comportamiento tienen más de un camino posible.

Cuando un proyecto llega a este punto, los errores raros empiezan a aparecer “por contexto” y no solo por bug explícito.

6. El monolito activo todavía concentra demasiado

Aunque funciona, `monitor.py` sigue siendo un archivo demasiado determinante.

Eso vuelve más difícil:

- corregir sin romper
- aislar responsabilidades
- probar por partes
- delegar mantenimiento

Qué Tan Bueno Está Hoy

Si tuviera que calificarlo con criterio práctico, no académico:

- Valor funcional: 8.5/10
- Apariencia para herramienta interna: 8/10
- Claridad de producto: 8/10
- Solidez arquitectónica actual: 6/10
- Mantenibilidad futura si no se consolida: 5.5/10
- Potencial real si se ordena: 9/10

Mi conclusión:

el software ya es bueno, pero su techo actual está limitado más por organización interna que por falta de talento o de idea.

Mejoras Recomendadas por Prioridad

Prioridad 1: Consolidar la verdad única del producto

Antes de agregar más funciones:

- definir cuál es la arquitectura oficial
- definir cuál es el nombre oficial del producto
- definir cuál es la versión real
- definir cuál es la fuente única de usuarios, configuración y métricas

Mientras eso no se cierre, cada mejora nueva aumenta deuda.

Prioridad 2: Cerrar la migración o frenarla explícitamente

Hay que elegir una de dos:

- terminar la migración al modelo modular
- o declarar temporalmente que el monolito sigue siendo la base oficial y congelar duplicaciones

Lo peor es seguir en tierra de nadie.

Prioridad 3: Unificar seguridad

La seguridad buena no debe quedarse en la rama vieja.

La rama nueva debería heredar:

- cifrado
- almacenamiento seguro
- esquema de autenticación fuerte

No al revés.

Prioridad 4: Sanear release y repositorio

Muy importante:

- resolver `installer.iss`
- limpiar artefactos y datos operativos del árbol de trabajo
- separar datos reales de software fuente
- evitar que archivos de runtime contaminen el proyecto

Prioridad 5: Validación y pruebas mínimas

No hablo de montar una NASA de testing. Hablo de blindar lo crítico:

- login
- alta/baja de equipos
- guardado/carga de configuración
- historial y uptime
- alertas
- empaquetado

Prioridad 6: Refinar UX final

Cuando lo anterior esté sólido:

- cerrar identidad visual
- bajar ruido visual en algunos textos
- hacer layout más adaptable
- pulir consistencia de branding y copy

Conclusión Final

Mi opinión sincera es que este software ya no es “una idea avanzada”; ya es una herramienta con valor operativo real.

Tiene criterio, intención, personalidad y una cantidad de funciones que demuestran horas de trabajo bien invertidas.

Lo que más me gusta es que se nota que fue construido desde un problema concreto y no desde una fantasía técnica.

Lo que más me preocupa no es la funcionalidad, sino la convivencia de varias versiones del mismo sistema al mismo tiempo.

Si ordenas esa parte, este proyecto puede pasar de "muy buena herramienta interna" a "producto serio y confiable".

Si no la ordenas, corres el riesgo clásico de los proyectos prometedores: seguir mejorando por fuera mientras por dentro crece una maraña difícil de sostener.

Mi cierre, sin adornos:

el software está bastante avanzado, está bien pensado y tiene futuro; pero ahora mismo necesita consolidación más que expansión.